

AI 训练集群网络均衡分担

time limit per test: 5 seconds
memory limit per test: 256 megabytes
input: standard input
output: standard output

Background

随着AI算力爆炸式增长，对AI集群算力规模和训练效率提出更高要求。AI训练场景具有流数量少、周期性，大流为主的特征，且并行phase间有强同步性要求，导致通信效率取决于最慢的节点，因此如何使参数面网络无阻塞，设备之间通信达到满吞吐成为了当前面临的挑战。

本挑战旨在设计一种端口分配算法，对于多个AI训练作业的参数同步通信，通过合理安排每条源宿流的上行出端口和上行入端口，使得不同的流在AI训练期间尽可能少的产生冲突。

Selection criteria

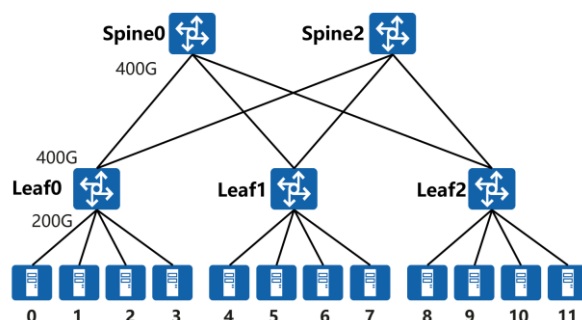
对于负载均衡问题

能够遵循输入/输出格式的规则，并与裁判系统进行交互。

能够设计端口分配算法，以在AI训练期间尽可能少的产生流冲突。

Task

下图是一个典型的Leaf-Spine组网，3台Leaf，每台Leaf都有2个400Gb/s的上行端口和4个200Gb/s的下行端口，每台Spine都有3个400Gb/s的端口，每台Leaf相同相对位置的上行端口都连接到同一个Spine上。有一个AI训练作业，共两个phase，每个phase都有4条卡到卡的流，需要给每个源宿流分配一个唯一的Leaf上行出端口。



Output中的port为ECMP算法所分配的出端口，对于Leaf内部的源宿流，port=-1表示不需要分配上行出端口。由于组网的特点，仅提供源Leaf上行出端口，即可确定唯一的宿Leaf的上行入端口。

下面以ECMP(equal cost multi path)算法示例，得到一个端口分配的方案：

Input			Output		
phase	src	dst	src	dst	port
phase0	0	1	0	1	-1
	1	4	1	4	1
	4	5	4	5	-1
	5	0	5	0	1
phase1	0	4	0	4	0
	1	5	1	5	1
	4	0	4	0	0
	5	1	5	1	1

对算法进行建模：对于一个Leaf-Spine组网，有 l 台型号相同的Leaf交换机和 p 台型号相同的Spine交换机。每台Leaf交换机的上行端口带宽都相同，下行端口带宽也相同，上行端口和下行端口的带宽比都 $r:1$ ，每台交换机有 p 个上行端口，有 $p \times r$ 个下行端口。所有的Leaf下行端口都连接了一张算力卡，第 i 台Leaf的第 j 个下行端口连接的算力卡的编号为 $p \times r \times i + j$ ， $i = 0, 1, 2 \dots l - 1$ ， $j = 0, 1, 2, \dots, r \times p - 1$ 。每台Spine交换机都有 l 个端口，且端口带宽都与Leaf交换机的上行端口带宽相同，所有的Leaf第 k 个上行端口都与第 k 台Spine上的一个端口连接。

Flow：一条源到宿卡的流。

Phase：一个AI训练作业中的一个阶段，包含多条源卡到宿卡的流。注意，在一个phase中，Job中所有的卡均仅会作为一条流的源卡和一条流的宿卡。

Job：一个AI训练作业，包含多个Phase。包含多张训练卡。

Job数量为 n ，分别为 $job_1, \dots, job_n$ ，对于每个Job，phase数量分别为 $m_1, \dots, m_n$ ，每个phase的流数分别为 $f_1, \dots, f_n$ ，第 t 个作业的第 m 个phase的第 f 条流的源卡为 $src_{t,m,f}$ ，宿卡为 $dst_{t,m,f}$ ，Leaf上行出端口为 $outport_{t,m,f}$ ，Leaf上行入端口为 $inport_{t,m,f}$ 。对于 Job_j 的 $Phase_m$ 的源为 s 的流，其上行出端口也可以表示为 $flowout_{j,m,s}$ 。

在实际训练中，由于一张卡在同一个时刻只会被一个AI训练任务使用，所以这张卡相关的流也只会存在于同一个Job中。

为了简化问题，我们假设每个AI训练任务只有一个通信域，且两张卡只会在一个phase中有通信关系。可以得到约束：一条源宿流，全局只会出现一次，即两个Job间不存在相同的源宿流，同一个Job内的两个Phase间不存在相同的源宿流。

Phase内的端口冲突比：如果在 Job_j 的 $Phase_m$ 内有 k 条流使用了Leaf的上行端口 i 的出方向，则在 Job_j 的 $Phase_m$ 内，上行端口 i 出方向的端口冲突比为 $\frac{k}{r}$ ；入方向同理。

Job内的端口冲突比： Job_j 内的上行端口出方向冲突比为 Job_j 中所有Phase内的上行端口出方向冲突比的最大值；入方向同理。

全局端口冲突比：全局上行端口出方向冲突比为所有Job的上行端口出方向的冲突比之和；入方向同理。

Phase内冲突：对于 Job_j 的 $Phase_m$ 的流 $Flow_f$ ，如果其上行出端口 $outport_{t,m,f}$ 在 Job_j 的 $Phase_m$ 内的上行端口出方向冲突比大于1或者其上行入端口 $inport_{t,m,f}$ 在 Job_j 的 $Phase_m$ 内的上行端口入方向冲突比大于1，则认为 Job_j 的 $Phase_m$ 的流 $Flow_f$ 存在Phase内冲突。

Phase间冲突：对于 Job_j 的 $Phase_m$ 的流 $Flow_f$ ，如果 Job_j 存在下一个Phase且该Phase中存在一条流与流 $Flow_f$ 共用相同的源卡，且这条流的上行出端口与流 $Flow_f$ 的上行出端口不相

同，则认为 Job_j 的 $Phase_m$ 的流 $Flow_f$ 存在 Phase间冲突。

Job间冲突：对于 Job_j 的 $Phase_m$ 的流 $Flow_f$ ，如果其上行出端口 $outport_{t,m,f}$ 在全局的上行端口出方向冲突比大于1或者其上行入端口 $inport_{t,m,f}$ 在全局的上行端口入方向冲突比大于1，则认为 Job_j 的 $Phase_m$ 的流 $Flow_f$ 存在Job间冲突。

以负载均衡为目标，算法的评分函数可以分为两个部分，第一部分为流的冲突评分，占比20%，分别统计了有Phase内冲突、Phase间冲突和Job间冲突的流数目，并求其加权分，三种冲突的权重依次递减；第二部分为最大端口负载得分，占比80%，分别从单Job维度和全局维度对最大端口负载进行了计算，并给出评分，两个维度的权重相同。

算法的目标可以建模为：

$$\begin{aligned}
 & \max \left(20 - \frac{12 * Cinphsc + 5 * Cbtphsc + 3 * Cbttskc}{\sum_{j=1}^n \sum_{m=1}^{m_j} \sum_{f=1}^{f_j} 1} + \frac{40}{Maxsingler} + \frac{40}{Maxmultir} \right) \\
 & Cinphsc = \sum_{j=1}^n \sum_{m=1}^{m_j} \sum_{f=1}^{f_j} Cinphs_{j,m,f} \\
 & Cbtphsc = \sum_{j=1}^n \sum_{m=1}^{m_j} \sum_{f=1}^{f_j} Cbtphs_{j,m,f} \\
 & Cbttskc = \sum_{j=1}^n \sum_{m=1}^{m_j} \sum_{f=1}^{f_j} Cbttsk_{j,m,f} \\
 & Cinphs_{j,m,f} = \begin{cases} 1, \sum_{f=1}^{f_j} sout_{j,m,f',outport_{j,m,f}} > r \text{ or } \sum_{f=1}^{f_j} sin_{j,m,f',inport_{j,m,f}} > r \\ 0, \sum_{f=1}^{f_j} sout_{j,m,f',outport_{j,m,f}} \leq r \text{ and } \sum_{f=1}^{f_j} sin_{j,m,f',inport_{j,m,f}} \leq r \end{cases} \\
 & Cbtphs_{j,m,f} = \begin{cases} 1, outport_{j,m,f} \neq flowout_{j,m+1,src_{j,m,f}} \text{ and } outport_{j,m,f} \neq -1 \text{ and } flowout_{j,m+1,src_{j,m,f}} \neq -1 \text{ and } m < m_j \\ 0, \text{ else} \end{cases} \\
 & Cbttsk_{j,m,f} = \begin{cases} 1, phsout_{j,m,f} + Maxexout_{j,outport_{j,m,f}} > r \text{ or } phsin_{j,m,f} + Maxexin_{j,outport_{j,m,f}} > r \\ 0, \text{ else} \end{cases} \\
 & phsout_{j,m,f} = \sum_{f=1}^{f_j} sout_{j,m,f',outport_{j,m,f}} \\
 & phsin_{j,m,f} = \sum_{f=1}^{f_j} sin_{j,m,f',inport_{j,m,f}}
 \end{aligned}$$

$$Maxexout_{j',i} = \sum_{j=1}^{j'-1} \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sout_{j,m,f,i} + \sum_{j=j'+1}^n \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sout_{j,m,f,i}$$

$$Maxexin_{j',i} = \sum_{j=1}^{j'-1} \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sin_{j,m,f,i} + \sum_{j=j'+1}^n \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sin_{j,m,f,i}$$

$$Maxmultir = \begin{cases} \frac{Maxmultic}{r}, \frac{Maxmultic}{r} > 1 \\ 1, \frac{Maxmultic}{r} \leq 1 \end{cases}$$

$$Maxsingler = \begin{cases} \frac{Maxsinglec}{r}, \frac{Maxsinglec}{r} > 1 \\ 1, \frac{Maxsinglec}{r} \leq 1 \end{cases}$$

$$Maxmultic = \max_{i \in \{0,1,\dots,p*r\}} (\max_{i \in \{0,1,\dots,p*r\}} Portoutm_i, \max_{i \in \{0,1,\dots,p*r\}} Portinm_i)$$

$$Maxsinglec = \max_{i \in \{0,1,\dots,p*r\}} (\max_{i \in \{0,1,\dots,p*r\}} Portouts_i, \max_{i \in \{0,1,\dots,p*r\}} Portins_i)$$

$$Portoutm_i = \sum_{j=1}^n \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sout_{j,m,f,i}$$

$$Portouts_i = \max_{j \in \{1,2,\dots,n\}} \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sout_{j,m,f,i}$$

$$sout_{j,m,f,i} = \begin{cases} 1, outport_{j,m,f} = i \text{ and } i \neq -1 \\ 0, outport_{j,m,f} \neq i \text{ or } i = -1 \end{cases}$$

$$Portinm_i = \sum_{j=1}^n \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sin_{j,m,f,i}$$

$$Portins_i = \max_{j \in \{1,2,\dots,n\}} \max_{m \in \{1,2,\dots,m_j\}} \sum_{f=1}^{f_j} sin_{j,m,f,i}$$

$$sin_{j,m,f,i} = \begin{cases} 1, inport_{j,m,f} = i \text{ and } i \neq -1 \\ 0, inport_{j,m,f} \neq i \text{ or } i = -1 \end{cases}$$

符号解释:

符号	解释
$Cinphsc$	Phase内冲突的流数量。
$Cbtphsc$	Phase间冲突的流数量。

$Cbttskc$	Job间冲突的流数量。
$Cinphs_{j,m,f}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 是否存在Phase内冲突。
$Cbtphs_{j,m,f}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 是否存在Phase间冲突。
$Cbttsk_{j,m,f}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 是否存在Job间冲突。
$phsout_{j,m,f}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 的上行出端口方向在 Job_j 的 $Phase_m$ 内的流数量。
$phsin_{j,m,f}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 的上行入端口方向在 Job_j 的 $Phase_m$ 内的流数量。
$Maxexout_{j,i}$	端口 i 在除 Job_j 外所有Job的Phase内上行端口出方向的最大流数量之和。
$Maxexin_{j,i}$	端口 i 在除 Job_j 外所有Job的Phase内上行端口入方向的最大流数量之和。
$Maxsingler$	所有Job中Phase中所有端口入和出方向的端口冲突比最大值之和的最大值，最小为1。
$Maxmultir$	所有Job中Phase中所有端口入和出方向的端口冲突比最大值，最小为1。
$Maxmultic$	所有Job中Phase中所有端口入和出方向的最大流数的最大值之和的最大值。
$Maxsinglec$	所有Job中Phase中所有端口入和出方向的最大流数的最大值。
$Portoutm_i$	对于所有Job先求出所有Phase中所有使用端口 i 上行端口出方向流数量的最大值再求和。
$Portouts_i$	所有Job的所有Phase中，所有使用端口 i 上行端口出方向流数量的最大值。
$sout_{j,m,f,i}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 的上行端口出方向是否为端口 i 。
$Portinm_i$	对于所有Job先求出所有Phase中所有使用端口 i 上行端口入方向流数量的最大值再求和。
$Portins_i$	所有Job的所有Phase中，所有使用端口 i 上行端口入方向流数量的最大值。
$sin_{j,m,f,i}$	Job_j 的 $Phase_m$ 流 $Flow_f$ 的上行端口入方向是否为端口 i 。

Input

在实际的多租户多任务AI训练场景中，在当前Job的端口规划时，是无法预知后续的Job细节的，因此多个Job的数据不会一次提供给算法，而是按照轮次分别提供，只有算法返回了上一轮Job的输出，才会进入下一轮并提供下一个Job的数据。这种交互会一直持续到算法返回最后一个Job的输出。

第1轮

第1行：作业数量 n ，leaf数量 l ，leaf上行端口数量 p ，leaf上下行端口带宽比 r

第2行：第一个作业的phase数量 m_1 ，每个phase的流数量 f_1

第3行：第一个phase的流 $src_{1,1,1}$ ， $dst_{1,1,1}$ ， $src_{1,1,2}$ ， $dst_{1,1,2}$ ，...， $src_{1,1,f_1}$ ， $dst_{1,1,f_1}$

...

第 $m_1 + 2$ 行：第 m_1 个phase的流 $src_{1,m_1,1}$ ， $dst_{1,m_1,1}$ ， $src_{1,m_1,2}$ ， $dst_{1,m_1,2}$... src_{1,m_1,f_1} ， dst_{1,m_1,f_1}

第2轮

...

第 n 轮

第1行：第 n 个作业的phase数量 m_n ，每个phase的流数量 f_n

第2行：第一个phase的流 $src_{n,1,1}$ ， $dst_{n,1,1}$ ， $src_{n,1,2}$ ， $dst_{n,1,2}$... $src_{n,1,f_n}$ ， $dst_{n,1,f_n}$

...

第 $m_n + 1$ 行：第 m_n 个phase的流 $src_{n,m_n,1}$ ， $dst_{n,m_n,1}$ ， $src_{n,m_n,2}$ ， $dst_{n,m_n,2}$... src_{n,m_n,f_n} ， dst_{n,m_n,f_n}

dst_{n,m_1,f_n}

其中作业数量 $n \leq 40$ ，leaf数量 $l \leq 100$ ，leaf上行端口数量 $p \leq 32$ ，leaf上下行端口带宽比 $r \leq 4$ ，单个作业的phase数量 $m_i \leq 31$ ，每个phase的流数量 $f_j \leq 12800$ 。

注意：在赛题中，每行不同数据间通过空格分隔。

Output

每轮都会按照相同格式输出

第 1 行，输出总流数 n

第 2 行，输出流出端口序列：

$src_1 \ dst_1 \ port_1 \ \dots \ src_n \ dst_n \ port_n$

其中 src_i ， dst_i ， $port_i$ 表示一条源卡为 src_i ，宿卡为 dst_i 的源宿流，其 Leaf 出端口为源卡接入 Leaf 的第 $port_i$ 个上行端口，Leaf 入端口为宿卡接入 Leaf 的第 $port_i$ 个上行端口。
在每轮输出中，对于重复出现的流，仅需要输出一次。

注意：输出结果中，每行不同数据间也请通过空格分隔。

Example1

Input

第一轮

```
2 2 2 2
1 4
2 3 3 6 6 7 7 2
```

第二轮

```
2 4
0 1 1 0 4 5 5 4
0 4 4 0 1 5 5 1
```

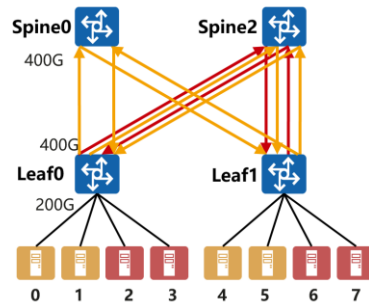
output

第一轮

```
4
2 3 -1 3 6 1 6 7 -1 7 2 1
```

第二轮

```
8
0 1 -1 1 0 -1 4 5 -1 5 4 -1 0 4 0 4 0 0 1 5 1 5 1 1
```



该输出没有pahse内冲突，没有phase间冲突，没有作业间冲突。得分为100。

Example2

Input

第一轮

3 3 2 2
1 4
0 4 4 0 1 5 5 1

第二轮

1 4
2 10 10 2 3 11 11 3

第三轮

1 4
6 8 8 6 7 9 9 7

Output1

第一轮

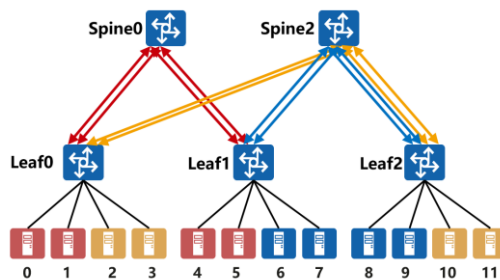
4
0 4 0 4 0 0 1 5 0 5 1 0

第二轮

4
2 10 1 10 2 1 3 11 1 11 3 1

第三轮

4
6 8 1 8 6 1 7 9 1 9 7 1



该输出没有pahse内冲突，没有phase间冲突，8条流有phase作业间冲突，单Job最大拥塞比为1，全局最大拥塞比为2。得分为 $20 \cdot 8 \cdot 3 / 12 + 40 / 1 + 40 / 2 = 78$ 。

Output2

第一轮

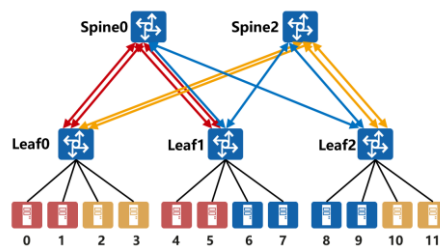
4
0 4 0 4 0 0 1 5 0 5 1 0

第二轮

4
2 10 1 10 2 1 3 11 1 11 3 1

第三轮

4
6 8 0 8 6 0 7 9 1 9 7 1



该输出没有pahse内冲突，没有phase间冲突，12条流有phase作业间冲突，单Job最大拥塞比为1，全局最大拥塞比为1.5。得分为 $20-12*3/12+40/1+40/1.5=83.7$ 。

Output3

第一轮

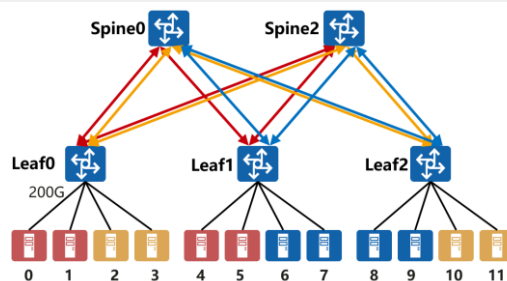
4
0 4 0 4 0 0 1 5 1 5 1 1

第二轮

4
2 10 0 10 2 0 3 11 1 11 3 1

第三轮

4
6 8 0 8 6 0 7 9 1 9 7 1



该输出没有pahse内冲突，没有phase间冲突，没有作业间冲突。得分为100。